Day 5. Multiple linear regression

Multiple features (variables)

Size in feet²	Number of bedrooms	Number of floors	Age of home in years	Price ($) in $1000's
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
…	…	…	…	…

Multiple features

Multiple features (variables)

	Size in feet²	Number of bedrooms	Number of floors	Age of home in years	Price ($) in $1000's
	x_1	x_2	x_3	x_4	
	2104	5	1	45	460
i=2	1416	3	2	40	232
	1534	3	2	30	315
	852	2	1	36	178
	…	…	…	…	…

$j = 1 \ldots 4$

$n = 4$

x_j = j^{th} feature

n = number of features

$\vec{x}^{(i)}$ = features of ith training example

$\vec{x}^{(2)} = [1416 \;\; 3 \;\; 2 \;\; 40]$

Model:

Previously: $f_{w,b}(x) = wx + b$

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + w_3 x_3 + w_4 x_4 + b$$

example

$$f_{w,b}(x) = 0.1 x_1 + 4 x_2 + 10 x_3 + -2 x_4 + 80$$

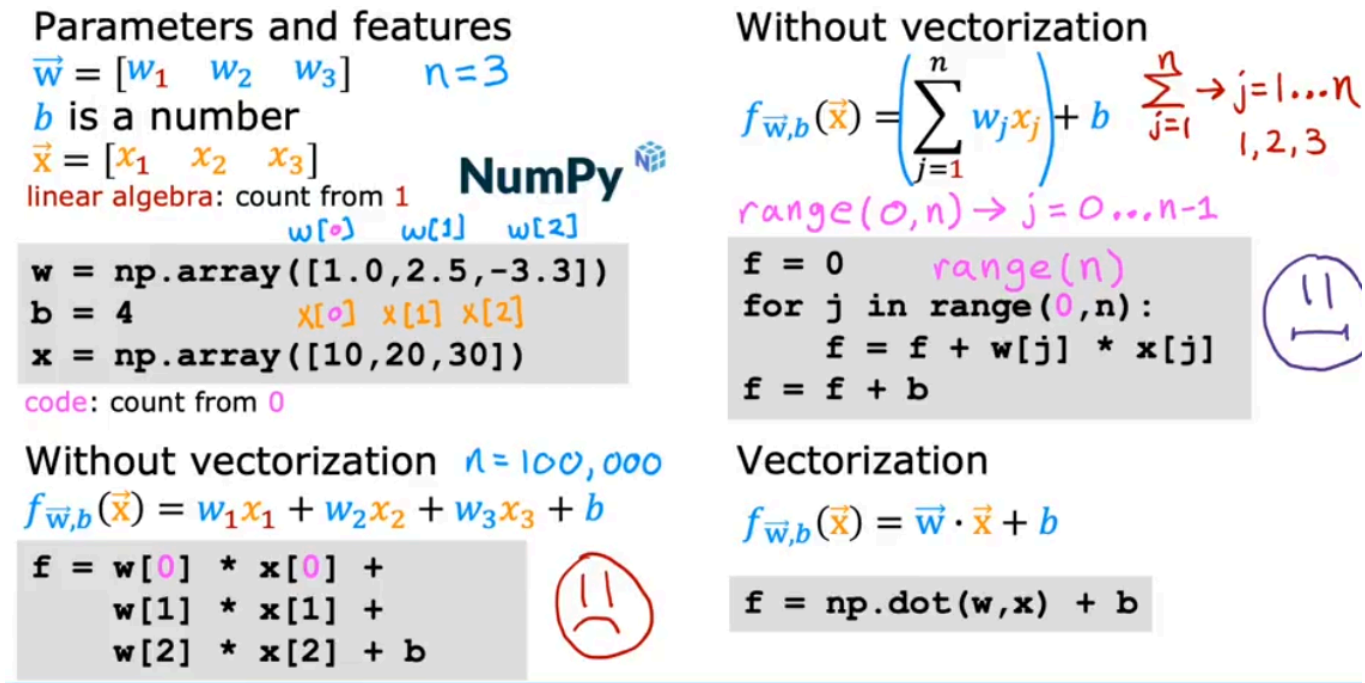
size, #bedrooms, #floors, years, base price

$$f_{w,b}(x) = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$

$$f_{w,b}(x) = \vec{w} \cdot \vec{x} + b$$

Vectorization

When you are learning algorightms, vectorization can shourten your code meanwhile improve the efficiency of the code.



ⓘ Why vectorization is faster

普通的 Python `for` 循环是在解释器层面逐个执行加法和乘法，一次只能处理一个数据。而 NumPy 的底层是 C 语言实现的，它利用了 CPU 的 **SIMD（Single Instruction, Multiple Data，单指令多数据流）** 指令集。

Gradient descent $\vec{w} = (w_1 \quad w_2 \quad \cdots \quad w_{16})$ ~~b~~ parameters

derivatives $\vec{d} = (d_1 \quad d_2 \quad \cdots \quad d_{16})$

```
w = np.array([0.5, 1.3, … 3.4])
d = np.array([0.3, 0.2, … 0.4])
```

learning rate α

compute $w_j = w_j - 0.1 d_j$ for $j = 1 \ldots 16$

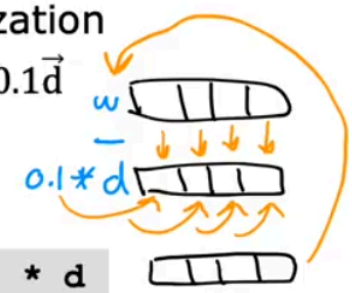
Without vectorization

$$w_1 = w_1 - 0.1 d_1$$
$$w_2 = w_2 - 0.1 d_2$$
$$\vdots$$
$$w_{16} = w_{16} - 0.1 d_{16}$$

```
for j in range(0,16):
    w[j] = w[j] - 0.1 * d[j]
```

With vectorization

$$\vec{w} = \vec{w} - 0.1\vec{d}$$




```
w = w - 0.1 * d
```